Recommender Systems for the Movie Lens Dataset

Nikita Mykytenko, Oleksandr Bobryshev, Labeat Dobraj

Academic Supervisor: Jun.-Prof. Dr. Dominik Stöger

Mathematical Institute for Machine Learning and Data Science

Catholic University of Eichstätt-Ingolstadt

August 15, 2026 Summer Semester 2026

Contents

1 Abstract **2**
- 1.1 Introduction . . . 3
- 1.2 Methodology . . . 3
- 1.3 Problem Statement . . . 4

2 Exploratory Data Analysis **5**
- 2.1 ml-latest-small . . . 5
- 2.2 ml-latest . . . 6

3 Machine Learning **7**
- 3.1 Spectral Matrix-Completion Models . . . 7
  - 3.1.1 Hard Thresholding . . . 7
  - 3.1.2 Soft Thresholding . . . 9
- 3.2 Matrix-Factorization Models . . . 10
  - 3.2.1 Gradients on the product matrix space . . . 10
  - 3.2.2 Gradient Descent . . . 11
  - 3.2.3 Stochastic Gradient Descent . . . 13
  - 3.2.4 Alternating Least Squares . . . 16

4 Results **19**
- 4.1 `ml-latest-small` . . . 19
  - 4.1.1 Hard Thresholding . . . 19
  - 4.1.2 Soft Thresholding . . . 20
  - 4.1.3 Hard versus Soft Thresholding . . . 20
  - 4.1.4 Gradient Descent . . . 21
  - 4.1.5 Stochastic Gradient Descent . . . 22
  - 4.1.6 Alternating Least Squares . . . 22
  - 4.1.7 Model Comparison . . . 23
- 4.2 `ml-latest` . . . 23
  - 4.2.1 Hard Thresholding . . . 23
  - 4.2.2 Soft Thresholding . . . 24
  - 4.2.3 Gradient Descent . . . 25
  - 4.2.4 Stochastic Gradient Descent . . . 26
  - 4.2.5 Alternating Least Squares . . . 26
  - 4.2.6 Model Comparison . . . 27

5 Difficulties **27**

6 Contributions **28**

7 Use of Artificial Intelligence **28**

References **29**

1 Abstract

The primary objective of our project is to implement and compare low-rank methods for predicting explicit movie ratings, and to determine which of them offers the best trade-off between predictive accuracy and computational scalability. We treat rating prediction as a matrix-completion problem: from a small, sparsely observed subset of a user–movie rating matrix R, we recover a prediction for any user–movie pair.

Two families of methods are compared. The first recovers R through spectral operations, repeatedly imputing the matrix and applying either hard thresholding (SVD truncation) or soft thresholding (singular-value shrinkage). The second fits the factorization $R \approx UV^\top$ directly to the mean-centred ratings, using full-batch gradient descent, stochastic gradient descent (SGD), and alternating least squares (ALS). Every model is tuned on a validation split and evaluated on a held-out 20 % test set using RMSE and MAE.

We conduct experiments on two MovieLens datasets of very different size: `ml-latest-small` with 100,836 ratings from 610 users on 9,724 movies (98.30 % sparse) and `ml-latest` with 33,832,162 ratings from 330,975 users on 83,239 movies (99.88 % sparse). On the small dataset, where a dense rating matrix still fits in memory, soft thresholding performs best with a test RMSE of 0.8703, a 17.0 % improvement over the global-mean baseline of 1.0488.

At full scale, a dense $330{,}975 \times 83{,}239$ matrix would require roughly 220 GB and is no longer an option. The dense thresholding implementations therefore drop out, but hard thresholding survives the jump in a matrix-free form that stores only the rank-r singular factors together with the sparse residual and never materializes the full grid; run on an HPC cluster, it reaches a test RMSE of 0.7996.

Sparse ALS achieves the lowest error overall with a test RMSE of 0.7931, a 25.5 % improvement over the baseline of 1.0642, followed by degree-scaled stochastic gradient descent at 0.7991 (24.9 %) and full-batch gradient descent at 0.8007. Soft thresholding was not carried over to full scale. Replacing the sparse NumPy ALS sweeps with parallel compiled kernels reduced the time per sweep from 16.47 s to 5.39 s, a speedup of 3.05× at an unchanged objective value, and compiling the SGD inner loop with `numba` was what made full-scale training practical at all.

Our results show that the decisive factor is not only the algorithm but also the representation it is implemented against: at full scale the four surviving methods lie within 0.008 RMSE of one another, so what separates them is not only how well they predict but also whether they can be run at all, and at what cost.

Keywords: Recommender Systems, Collaborative Filtering, Matrix Completion, Matrix Factorization, Singular Value Thresholding, Alternating Least Squares, Stochastic Gradient Descent, Sparse Linear Algebra, MovieLens, RMSE

1.1 Introduction

Recommender systems have become a quiet backbone of everyday digital life. Netflix decides which shows appear on your homepage, Amazon predicts which products you are likely to buy next, and Spotify curates playlists tailored to individual listening habits. What all these systems have in common is a deceptively simple underlying task: given a partially observed matrix of user preferences, in the form of clicks, purchases, or explicit ratings, predict the missing entries as accurately as possible. The commercial relevance of solving this task well was made highly visible by the Netflix Prize, a public competition that awarded one million US dollars to the first team that improved Netflix's own recommendation algorithm by 10 %, and that popularized matrix factorization as the dominant approach to collaborative filtering.

The central difficulty in this task is sparsity. A typical user rates only a tiny fraction of the items available on a platform, so the observed user–item matrix is overwhelmingly incomplete: most entries are simply unknown rather than zero. Recovering a full, dense prediction from such a sparse observation is a matrix-completion problem, and it only becomes solvable if one assumes that the true, unobserved matrix has additional structure, most commonly that it is approximately low-rank. Intuitively, this means that a person's taste in movies, or the appeal of a given movie across users, can be summarized by a small number of latent factors rather than by treating every user–item pair independently.

The theoretical background of this project was developed by reviewing the overview articles and lecture notes provided by Prof. Dominik Stöger. In particular, we relied on the supplied literature on low-rank matrix recovery, singular-value decomposition, and nonconvex matrix factorization [1, 2, 3].

Before implementing the models, we performed an exploratory analysis of both MovieLens datasets. We examined their dimensions, sparsity, rating distributions, and the distribution of observed ratings across users and movies. This analysis also showed why dense implementations can be used for `ml-latest-small`, whereas sparse or matrix-free implementations are required for `ml-latest`.

Two broad families of methods exploit this low-rank assumption. The first imputes the missing entries and repeatedly applies a spectral operation to the resulting matrix, for example through truncated singular-value decomposition or nuclear-norm proximal steps. The second bypasses the dense matrix entirely and instead fits a low-rank factorization directly to the observed entries, typically trained with gradient-based or alternating optimization methods. Both families come with their own trade-offs between predictive accuracy, computational cost, and how well they scale once the number of users and items grows into the millions.

In this project, we implement and compare both families of methods for the task of predicting explicit movie ratings, and we use two MovieLens datasets of very different size to study how their trade-offs shift with scale. The remainder of this report outlines our methodology, states the problem more precisely, and reports our findings.

1.2 Methodology

All experiments were implemented in Python, using `numpy` and `pandas` for data handling, `scipy` and `scikit-learn` for linear algebra and baseline utilities, `matplotlib` for visualization, and `numba` to just-in-time compile the training loops of the factorization models once a dense rating matrix was no longer feasible.

For every dataset, we first split the observed ratings into a training set (80 %) and a held-out test set (20 %), which is only ever touched for the final evaluation of a fully tuned model. For hyperparameter tuning, the training set is split further into a fit set (80 %) and a validation set (20 %): models are trained on the fit set for a range of hyperparameter configurations, and the configuration with the lowest validation error is selected before the model is retrained on the full training set and evaluated once on the test set. For `ml-latest-small`, this results in 64,534 fit,

16,134 validation, and 20,168 test ratings; for `ml-latest`, the equivalent split contains 21,652,583 fit, 5,413,146 validation, and 6,766,433 test ratings.

Model quality is measured with two metrics computed on clipped predictions $\hat{r} \in [0.5, 5.0]$:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(r_i - \hat{r}_i)^2}\,, \qquad \text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|r_i - \hat{r}_i|\,,$$

where:

- n is the number of rated user–movie pairs being evaluated,
- r_i is the true rating,
- $\hat{r}_i$ is the model's predicted rating, clipped to the valid rating range $[0.5, 5.0]$ before the error is computed.

RMSE penalizes large individual errors more heavily and is used to select the best hyperparameter configuration during tuning, while MAE reports the typical absolute error in the original rating scale and is used alongside RMSE to interpret the final test results.

1.3 Problem Statement

Suppose there are m users and n movies. Ideally, every user–movie pair would carry a rating, forming a complete matrix

$$R^{\text{true}} \in \mathbb{R}^{m\times n}, \qquad (R^{\text{true}})_{ij} = \text{rating of user } i \text{ for movie } j.$$

In practice, only a small subset of entries is ever recorded. We write

$$\Omega \subseteq \{1,\dots,m\} \times \{1,\dots,n\}$$

for the set of observed user–movie pairs, so that R is known only on Ω. Everywhere outside Ω the true rating is missing, not zero, and inferring it is the problem this project solves.

Definition 1.1 (Observation operator)**.** *For $Z \in \mathbb{R}^{m\times n}$, define*

$$[P_\Omega(Z)]_{ij} = \begin{cases} Z_{ij}, & (i,j) \in \Omega, \\ 0, & (i,j) \notin \Omega. \end{cases}$$

$P_\Omega(\cdot)$ keeps the observed entries of Z and sets every unobserved entry to zero.

The methods compared in this report address the same underlying task of learning a low-rank representation from the observed ratings, but they impose the low-rank structure in different ways. Hard thresholding is based directly on the rank-constrained matrix-completion problem

$$\min_{X\in\mathbb{R}^{m\times n}} \|P_\Omega(X - R)\|_F^2 \qquad \text{subject to} \qquad \text{rank}(X) \le r,$$

where:

- X is the completed rating matrix the algorithm is trying to learn,
- R is the partially observed matrix of true ratings, known only on Ω,
- $\|\cdot\|_F$ is the Frobenius norm, so the objective penalizes the squared error only at the observed entries,

- r is the target rank, reflecting the assumption that user preferences can be explained by a small number of latent factors.

Soft thresholding does not directly solve this rank-constrained problem. It replaces the explicit rank constraint with a nuclear-norm penalty. The matrix-factorization models instead represent the prediction matrix as $X \approx UV^\top$ and optimize the user and movie factors directly. Thus, the methods use different objectives and optimization procedures, but all of them exploit low-rank structure to predict the unobserved ratings.

Once training is complete, the entry X_{ij} is the model's predicted rating for user i on movie j, whether or not that pair was ever observed.

2 Exploratory Data Analysis

Before any model is trained, it is important to understand the structure of the data itself, since the same modeling choices can behave very differently depending on how large and how sparse the underlying rating matrix is. We therefore examine the two MovieLens datasets used throughout this report separately: `ml-latest-small` (Section 2.1), a development-scale benchmark small enough to fit as a dense matrix and iterate on quickly, and `ml-latest` (Section 2.2), the full-scale dataset our final models are judged on. For both, we check for missing values and duplicates, examine the distribution of ratings, and quantify how sparse the observed user–movie matrix actually is.

2.1 ml-latest-small

The `ml-latest-small` dataset contains 100,836 ratings from 610 users on 9,724 distinct movies, with values on the half-star scale from 0.5 to 5.0. The accompanying movie metadata lists 9,742 movies in total, so 18 movies carry no rating at all and never appear in the ratings file. Movie identifiers are also not contiguous: they range from 1 to 193,609, but only 9,742 of the 193,609 possible IDs are actually assigned, so any downstream matrix representation has to re-index movies rather than use the raw ID as a column index directly. The dataset is clean in the strict sense: neither the ratings table nor the movies table contains missing values or duplicate rows.

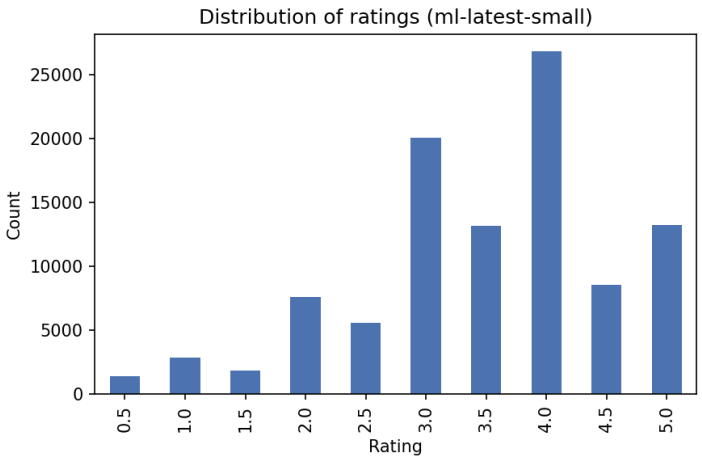


Figure 1: Distribution of the 100,836 ratings in `ml-latest-small`.

The rating distribution (Figure 1) is far from uniform. A rating of 4.0 is the single most common value with 26,818 occurrences, followed by 3.0 (20,047) and 5.0 (13,211); the mean rating is 3.50. Whole-star ratings are consistently more frequent than the neighboring half-star ratings (e.g. 4.0 outnumbers both 3.5 and 4.5), which suggests that users gravitate toward round

numbers even though the platform allows finer granularity. Low ratings are comparatively rare: 0.5, the lowest possible value, was given only 1,370 times.

Relating the number of observed ratings to the $610 \times 9{,}724$ user–movie grid gives a sparsity of 98.30 %, i.e. only 1.70 % of all possible user–movie pairs are observed. Ratings are also concentrated on a small set of popular movies: the ten most rated titles, led by *Forrest Gump* (329 ratings), *The Shawshank Redemption* (317), and *Pulp Fiction* (307), each received far more ratings than a typical movie in the long tail, which is consistent with the low-rank structure assumed in Section 1.3

2.2 ml-latest

The `ml-latest` dataset is the full-scale counterpart to the small benchmark and the dataset our production candidates are ultimately judged on. It contains 33,832,162 ratings from 330,975 users on 83,239 distinct movies, again on the 0.5–5.0 half-star scale. The movie metadata lists 86,537 movies in total, so 3,298 movies carry no rating; movie IDs range from 1 to 288,983, of which only 86,537 are actually assigned. As with the small dataset, neither the ratings table nor the movies table contains missing values or duplicate rows.

	`ml-latest-small`	`ml-latest`
Ratings	100,836	33,832,162
Users	610	330,975
Rated movies	9,724	83,239
Sparsity	98.30 %	99.88 %

Table 1: `ml-latest` holds 336× more ratings than `ml-latest-small`, drawn from a user–movie grid that is more than 4,600× larger.

Table 1 makes the jump in scale explicit. Relating the 33.8 million observed ratings to the full $330{,}975 \times 83{,}239$ user–movie grid gives a sparsity of 99.88 %, so only 0.12 % of all possible user–movie pairs are observed, against 1.70 % for the small dataset: the full dataset is not just larger, it is proportionally far sparser.

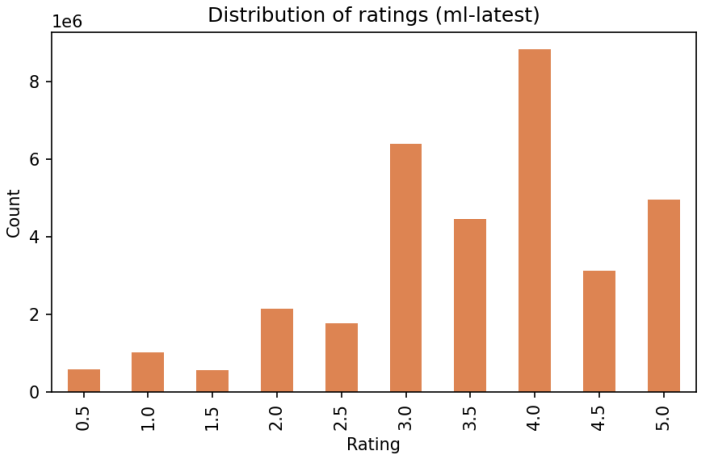


Figure 2: Distribution of the 33,832,162 ratings in `ml-latest`.

The rating distribution (Figure 2) mirrors the pattern seen in the small dataset: 4.0 is again the most common rating with 8,835,955 occurrences, followed by 3.0 (6,400,664) and 5.0 (4,957,902); the mean rating is 3.54, close to the 3.50 mean observed in the small dataset. Whole-star ratings again dominate their neighboring half-star values, and the two most popular titles

at full scale, *The Shawshank Redemption* (122,296 ratings) and *Forrest Gump* (113,581 ratings), are the same two movies that led the small dataset, just with two orders of magnitude more ratings each.

This 0.12 % observation density has a direct computational consequence. A dense 330,975 × 83,239 matrix of `float64` entries would require

$$330{,}975 \times 83{,}239 \times 8 \text{ bytes} \approx 220 \text{ GB}.$$

This motivates the sparse and matrix-free implementations of hard thresholding used for the full-scale dataset.

3 Machine Learning

Before comparing any of the low-rank methods introduced in Section 1.3, we first establish a simple reference point. The baseline model ignores users and movies entirely and predicts the same value, the global mean rating of the training set, for every user–movie pair:

$$\mu = \frac{1}{|\Omega_{\text{train}}|} \sum_{(i,j)\in\Omega_{\text{train}}} R_{ij}, \qquad \hat{r}_{ij} = \mu \quad \text{for every } (i,j).$$

This model has no latent structure and no per-user or per-movie adjustment, so it represents the error we would expect from knowing nothing about individual taste. Any model with actual predictive value has to beat it; a method that only matches the baseline is not learning anything useful from the observed ratings.

Dataset	μ	Test RMSE	Test MAE
`ml-latest-small`	3.5026	1.0488	0.8316
`ml-latest`	3.5425	1.0642	0.8431

Table 2: Global-mean baseline performance on both datasets. All models in the remainder of this section are evaluated against these numbers.

The baseline already performs similarly on both datasets, which is expected since it only depends on the overall rating distribution and not on the sparsity or size of the matrix. The following subsections introduce hard thresholding, soft thresholding, gradient descent, stochastic gradient descent, and alternating least squares, and show how far each of them improves on the numbers in Table 2.

3.1 Spectral Matrix-Completion Models

We consider two spectral matrix-completion methods: hard thresholding and soft thresholding.

3.1.1 Hard Thresholding

Rank-constrained objective. Hard thresholding uses the rank-constrained matrix-completion problem

$$\min_{X\in\mathbb{R}^{m\times n}} \frac{1}{2}\|P_\Omega(X-R)\|_F^2 \qquad \text{subject to} \qquad \operatorname{rank}(X) \le r. \tag{3.1}$$

The loss measures the reconstruction error on the observed ratings, while the constraint allows at most r singular components. Hence, r directly controls the complexity of the completed matrix.

For an economy SVD

$$Z = U \operatorname{diag}(\sigma_1, \ldots, \sigma_q) V^\top, \qquad \sigma_1 \geq \cdots \geq \sigma_q \geq 0,$$

the hard-thresholding operator keeps the r largest singular values and sets all remaining values to zero:

$$H_r(Z) = \sum_{k=1}^{r} \sigma_k u_k v_k^\top. \tag{3.2}$$

The retained singular values are not changed. Truncating the SVD after the first r components gives the best rank-r approximation to Z in the Frobenius norm [2]. Combining this projection with the completion step gives the update

$$X^{(t+1)} = H_r\Big(P_\Omega(R) + P_{\Omega^c}\Big(X^{(t)}\Big)\Big). \tag{3.3}$$

This is an iterative hard-thresholding update for low-rank matrix recovery [1].

Dense implementation. In `models.py`, $X^{(0)}$ is initialized with the mean of the observed training ratings. At each iteration, the implementation inserts the known ratings, computes

`np.linalg.svd(..., full_matrices=False)`,

and reconstructs the matrix using the first r singular triplets. It then applies

$$X^{(t+1)} \leftarrow \operatorname{clip}\Big(X^{(t+1)}, 0.5, 5.0\Big).$$

Clipping enforces the rating range used by the project. It is separate from the rank constraint in Equation (3.1).

Computational and memory cost. Let $q = \min(m, n)$. The dense implementation stores the completed matrix, the imputed matrix, and the SVD factors. Its memory consumption is therefore dominated by $O(mn)$ dense storage plus the SVD workspace.

A full economy SVD costs $O(mnq)$ operations per iteration. Reconstructing the rank-r approximation adds $O(mnr)$. For T iterations, the SVD dominates the total computational cost, which is approximately

$$O(Tmnq).$$

Matrix-free implementation for the full data set. As discussed in Section 2.2, the full `ml-latest` user–movie matrix is too large to store densely.

The matrix-free implementation stores the low-rank iterate through compact singular factors and computes only a partial SVD during each projection step [3].

The current rank-r iterate is represented as

$$X^{(t)} = U_t \Sigma_t V_t^\top.$$

The training ratings are stored in a sparse matrix. Predictions are computed only for the observed user–movie pairs. Their residuals form the sparse correction

$$C^{(t)} = P_\Omega\Big(R - X^{(t)}\Big).$$

The matrix required by the hard-thresholding step is represented as

$$Z^{(t)} = U_t \Sigma_t V_t^\top + C^{(t)}.$$

At observed positions, this expression equals the true rating. At missing positions, it equals the current prediction. It therefore represents the same imputation matrix as Equation (3.3), but the complete dense matrix is never materialized.

A `LinearOperator` provides products with $Z^{(t)}$ and $(Z^{(t)})^\top$ to `scipy.sparse.linalg.svds`. The solver computes only the r largest singular triplets, which become the factors for the next iteration. This changes the storage and SVD procedure, but not the underlying hard-thresholding update.

If $|\Omega|$ denotes the number of observed ratings, the matrix-free memory cost is

$$O(|\Omega| + r(m+n)).$$

It stores the sparse ratings and correction together with the two factor matrices. Predictions and residuals cost $O(|\Omega|r)$, while one product with the virtual matrix costs

$$O(|\Omega| + r(m+n)).$$

If the iterative SVD requires N_{mv} matrix-vector products, one outer iteration costs approximately

$$O\big(|\Omega|r + N_{\text{mv}}\big(|\Omega| + r(m+n)\big)\big).$$

The runtime depends on ARPACK convergence, but this implementation replaces the $O(mn)$ memory bottleneck with repeated sparse passes over the observed ratings. Clipping is used only for metric calculation, so the stored factors remain exactly rank r. This makes a full-data-set run feasible on Fritz without materializing the complete user–movie grid.

3.1.2 Soft Thresholding

Nuclear-norm objective. Soft thresholding replaces the fixed rank constraint with a nuclear-norm penalty [1]:

$$\min_{X\in\mathbb{R}^{m\times n}} F_\lambda(X) = \frac{1}{2}\|P_\Omega(X-R)\|_F^2 + \lambda\|X\|_*, \qquad \|X\|_* = \sum_{k=1}^{q}\sigma_k(X). \tag{3.4}$$

The first term fits the observed ratings. The second penalizes the sum of the singular values and acts as a convex surrogate for rank [1]. Increasing λ places more weight on a simpler matrix, whereas decreasing it places more weight on the observed-data fit.

The proximal operator of the nuclear norm is singular-value soft thresholding [1]:

$$S_\tau(Z) = U\,\mathrm{diag}((\sigma_k-\tau)_+)\,V^\top, \qquad (a)_+ = \max(a,0). \tag{3.5}$$

Unlike hard thresholding, this operator reduces every retained singular value by τ. Values at or below τ become zero. Consequently, the effective rank is determined by the current spectrum instead of being fixed beforehand.

For a proximal-gradient step size η, the threshold is

$$\tau = \eta\lambda.$$

The project uses the unit-step completion formula. Therefore, $\tau = \lambda$, and the update before clipping is

$$X^{(t+1)} = S_\tau\Big(P_\Omega(R) + P_{\Omega^c}(X^{(t)})\Big). \tag{3.6}$$

Implementation. The implementation applies Equation (3.5) through

```
np.maximum(s - tau, 0)
```

after computing the economy SVD. It uses the same global-mean initialization and $[0.5, 5.0]$ clipping as the hard-thresholding model.

The value of τ controls both the shrinkage of the retained singular values and the effective rank of the reconstructed matrix. A large value removes more singular components and produces a simpler model. A small value retains more components and allows the model to fit the observed ratings more closely.

Computational and memory cost. The current soft-thresholding implementation materializes dense matrices X and Z and computes a full economy SVD at every iteration. Its asymptotic memory cost is therefore $O(mn)$ plus the SVD workspace.

Its dominant computational cost is $O(mnq)$ per iteration, where $q = \min(m, n)$. For T iterations, the total computational cost is approximately

$$O(Tmnq).$$

The shrinkage operation itself costs only $O(q)$. Consequently, the computational cost comes almost entirely from repeatedly computing the dense SVD rather than from subtracting τ from the singular values [1].

3.2 Matrix-Factorization Models

Matrix factorization enforces the rank constraint implicitly by writing $X = UV^\top$, where $U \in \mathbb{R}^{m\times r}$ contains user factors and $V \in \mathbb{R}^{n\times r}$ contains movie factors. Gradient descent, stochastic gradient descent, and ALS are based on the following regularized objective. For SGD, the objective is additionally scaled by $1/N_\Omega$, where $N_\Omega = |\Omega|$.

$$\begin{aligned} F_\lambda(U,V) &= \frac{1}{2}\sum_{(u,i)\in\Omega}\left(r_{ui} - \mu - \boldsymbol{u}_u^\top \boldsymbol{v}_i\right)^2 + \frac{\lambda}{2}\left(\|U\|_F^2 + \|V\|_F^2\right) \\ &= \frac{1}{2}\|P_\Omega(UV^\top - Y)\|_F^2 + \frac{\lambda}{2}\left(\|U\|_F^2 + \|V\|_F^2\right). \end{aligned} \tag{3.7}$$

where μ is the global training mean, Y is defined as $Y_{ui} := R_{ui} - \mu$, and $\lambda \geq 0$ controls regularization.

3.2.1 Gradients on the product matrix space

In discussing the optimization algorithms below, we repeatedly differentiate functions on a product of matrix spaces. We therefore record the required definitions.

Definition 3.1 (Gradient on a Euclidean space)**.** *Let $\mathcal{E}$ be a finite-dimensional real vector space equipped with an inner product $\langle\cdot,\cdot\rangle_\mathcal{E}$, and let $f : \mathcal{E} \to \mathbb{R}$ be smooth. The differential of f at $x \in \mathcal{E}$ is the linear map $\mathrm{D}f(x) : \mathcal{E} \to \mathbb{R}$ characterized by*

$$\mathrm{D}f(x)[h] = \lim_{t\to 0}\frac{f(x+th) - f(x)}{t}.$$

The gradient of f at x, with respect to this inner product, is the unique vector $\nabla_\mathcal{E} f(x) \in \mathcal{E}$ such that

$$\mathrm{D}f(x)[h] = \langle h, \nabla_\mathcal{E} f(x)\rangle_\mathcal{E} \qquad \text{for every } h \in \mathcal{E}.$$

See, e.g., Boumal [4, Sec. 3.1, p. 24].

For matrix factorization, the parameter space is $\mathcal{P} = \mathbb{R}^{m\times r} \times \mathbb{R}^{n\times r}$, equipped with the product Frobenius inner product

$$\langle (A,B),(C,D)\rangle_{\mathcal{P}} := \langle A,C\rangle_F + \langle B,D\rangle_F, \qquad \langle A,C\rangle_F := \operatorname{tr}(A^\top C).$$

For differentiable $F_\lambda : \mathcal{P} \to \mathbb{R}$, its product-space gradient is the pair of block gradients

$$\nabla_{\mathcal{P}} F_\lambda(U,V) := (\nabla_U F_\lambda(U,V), \nabla_V F_\lambda(U,V)) \in \mathcal{P}. \tag{3.8}$$

Applying Definition 3.1 with $\mathcal{E} = \mathcal{P}$ gives, for every $(A,B) \in \mathcal{P}$,

$$\begin{aligned} \mathrm{D}F_\lambda(U,V)[A,B] &= \langle \nabla_{\mathcal{P}} F_\lambda(U,V), (A,B)\rangle_{\mathcal{P}} \\ &= \langle \nabla_U F_\lambda(U,V), A\rangle_F + \langle \nabla_V F_\lambda(U,V), B\rangle_F . \end{aligned} \tag{3.9}$$

In particular, the entries of the block gradients are the corresponding partial derivatives:

$$(\nabla_U F_\lambda(U,V))_{uk} = \frac{\partial F_\lambda}{\partial U_{uk}}(U,V), \qquad (\nabla_V F_\lambda(U,V))_{ik} = \frac{\partial F_\lambda}{\partial V_{ik}}(U,V). \tag{3.10}$$

In particular, see Beck [5, Remark 3.32].

The following short-hand definition will also be used repeatedly throughout this chapter:

Definition 3.2 (Local observation sets)**.** *For each user $u \in \{1,\dots,m\}$, define*

$$\Omega_u := \{i \in \{1,\dots,n\} : (u,i) \in \Omega\}. \tag{3.11}$$

For each movie $i \in \{1,\dots,n\}$, define

$$\Omega^i := \{u \in \{1,\dots,m\} : (u,i) \in \Omega\}. \tag{3.12}$$

3.2.2 Gradient Descent

Using the results stated above, we see that $\left(U^{t+1}, V^{t+1}\right) = \left(U^t, V^t\right) - \eta \nabla_{\mathcal{P}} F_\lambda \left(U^t, V^t\right)$ is equivalent to

$$\begin{aligned} U^{t+1} &= U^t - \eta \nabla_U F_\lambda(U_t, V_t), \\ V^{t+1} &= V^t - \eta \nabla_V F_\lambda(U_t, V_t). \end{aligned} \tag{3.13}$$

We now state the general gradient descent algorithm.

Algorithm 1 Gradient descent for matrix factorization

Require: Learning rate $\eta > 0$ and number of iterations T

1: Initialize $U_0 \in \mathbb{R}^{m\times r}$ and $V_0 \in \mathbb{R}^{n\times r}$ with small random entries
2: **for** $t = 0, 1, \dots, T-1$ **do**
3: $\quad U_{t+1} \leftarrow U_t - \eta \nabla_U F_\lambda(U_t, V_t)$
4: $\quad V_{t+1} \leftarrow V_t - \eta \nabla_V F_\lambda(U_t, V_t)$
5: **end for**
6: **return** U_T, V_T

Direct differentiation of the objective in Equation 3.7 gives the following gradient formulas; cf. Hastie et al. [6, Eq. (6), Secs. 5.2–5.3.1].

Theorem 3.3 (Gradients of the regularized factorization objective)**.** *Let $\lambda \geq 0$. The objective F_λ in Equation 3.7 is differentiable on $\mathcal{P}$. Its block gradients are*

$$\begin{aligned} \nabla_U F_\lambda(U,V) &= P_\Omega(UV^\top - Y)V + \lambda U, \\ \nabla_V F_\lambda(U,V) &= P_\Omega(UV^\top - Y)^\top U + \lambda V. \end{aligned} \tag{3.14}$$

The following corollary gives row-wise formulas for evaluating the gradients from the observed ratings, thus avoiding forming a dense $m \times n$ residual matrix.

Corollary 3.4 (Row-wise gradients)**.** *For every user $u \in \{1, \dots, m\}$ and movie $i \in \{1, \dots, n\}$,*

$$[\nabla_U F_\lambda(U, V)]_{u,:} = \sum_{j \in \Omega_u} \left(U_{u,:} V_{j,:}^\top - Y_{uj} \right) V_{j,:} + \lambda U_{u,:}, \tag{3.15}$$

$$[\nabla_V F_\lambda(U, V)]_{i,:} = \sum_{a \in \Omega^i} \left(U_{a,:} V_{i,:}^\top - Y_{ai} \right) U_{a,:} + \lambda V_{i,:}. \tag{3.16}$$

With these results in place, we can rewrite Algorithm 1.

Algorithm 2 Gradient descent

Require: Observed centered ratings Y_{ui} for $(u, i) \in \Omega$; local sets $(\Omega_u)_{u=1}^m$ and $(\Omega^i)_{i=1}^n$; latent dimension r; regularization parameter λ; iteration count $T \in \mathbb{N}$; step size $\eta > 0$; initial factors $(U^{(0)}, V^{(0)}) \neq (0, 0)$

for $t = 0, \dots, T-1$ **do**
  for $u = 1, \dots, m$ **do**
    $\left(G_U^{(t)}\right)_{u,:} \leftarrow \sum_{j \in \Omega_u} \left(U_{u,:}^{(t)} \left(V_{j,:}^{(t)}\right)^\top - Y_{uj} \right) V_{j,:}^{(t)} + \lambda U_{u,:}^{(t)}$
  end for
  for $i = 1, \dots, n$ **do**
    $\left(G_V^{(t)}\right)_{i,:} \leftarrow \sum_{a \in \Omega^i} \left(U_{a,:}^{(t)} \left(V_{i,:}^{(t)}\right)^\top - Y_{ai} \right) U_{a,:}^{(t)} + \lambda V_{i,:}^{(t)}$
  end for
  for $u = 1, \dots, m$ **do**
    $U_{u,:}^{(t+1)} \leftarrow U_{u,:}^{(t)} - \eta \left(G_U^{(t)}\right)_{u,:}$
  end for
  for $i = 1, \dots, n$ **do**
    $V_{i,:}^{(t+1)} \leftarrow V_{i,:}^{(t)} - \eta \left(G_V^{(t)}\right)_{i,:}$
  end for
end for
return $U^{(T)}, V^{(T)}$

Our implementation corresponds Algorithm 2. We do not form the full $m \times n$ residual matrix $P_\Omega \left(U^{(t)} \left(V^{(t)}\right)^\top - Y \right)$. At iteration t, the gradient matrices are initialized with the regularization terms,

$$G_U^{(t)} \leftarrow \lambda U^{(t)}, \qquad G_V^{(t)} \leftarrow \lambda V^{(t)}.$$

Then, for every observed pair $(u, i) \in \Omega$, the implementation computes the residual

$$e_{ui}^{(t)} = U_{u,:}^{(t)} \left(V_{i,:}^{(t)}\right)^\top - Y_{ui}$$

and adds its contributions to the corresponding rows of both block gradients:

$$\left(G_U^{(t)}\right)_{u,:} \leftarrow \left(G_U^{(t)}\right)_{u,:} + e_{ui}^{(t)} V_{i,:}^{(t)},$$
$$\left(G_V^{(t)}\right)_{i,:} \leftarrow \left(G_V^{(t)}\right)_{i,:} + e_{ui}^{(t)} U_{u,:}^{(t)}.$$

After all observed pairs have been processed, the resulting matrices satisfy

$$G_U^{(t)} = \nabla_U F_\lambda \left(U^{(t)}, V^{(t)} \right), \qquad G_V^{(t)} = \nabla_V F_\lambda \left(U^{(t)}, V^{(t)} \right).$$

Then, we update the user and movie factors as in Algorithm 2.

Sparse implementation with Numba. The observed ratings are stored in a CSR matrix, and the GD loops are compiled with `Numba`. Unlike ALS, GD does not use a CSC matrix or `prange`, because several ratings may contribute to the same gradient row.

Computational complexity of our implementation. Initializing the two gradient matrices with the regularization terms and updating both factor matrices require $\mathcal{O}((m+n)r)$ operations per iteration. For each of the $N_\Omega = |\Omega|$ observed ratings, computing the length-r inner product and accumulating its contributions to both gradients requires $\mathcal{O}(r)$ operations. Hence, one iteration costs

$$\mathcal{O}\left(\big(N_\Omega + m + n\big)r\right),$$

and T iterations cost

$$\mathcal{O}\left(T\big(N_\Omega + m + n\big)r\right).$$

Memory complexity of our implementation. The observed ratings and their user and movie indices require $\mathcal{O}(N_\Omega)$ storage, while the factors and gradient matrices require $\mathcal{O}((m+n)r)$ storage. Therefore, the total memory complexity is

$$\mathcal{O}\left(N_\Omega + (m+n)r\right).$$

Remark 3.5 (Comparison with a dense implementation)**.** *A dense implementation that explicitly forms the matrices $U^{(t)}(V^{(t)})^\top$ and $P_\Omega(U^{(t)}(V^{(t)})^\top - Y)$ requires $\mathcal{O}(mnr)$ operations per iteration and $\mathcal{O}(mn + (m+n)r)$ memory. Our sparse implementation replaces the dependence on the total number mn of possible user–movie pairs by the number $N_\Omega = |\Omega|$ of observed ratings.*

Note on the importance of step-size. From the gradient-descent theory discussed in the Foundations of Machine Learning course, we know that at any non-stationary point there exists a sufficiently small step size $\eta > 0$ for which a step in the negative-gradient direction decreases the objective. Hence, if a fixed step size produces oscillations or repeated increases of the training objective, this suggests that η is too large for the corresponding iterates. We observe this behaviour for one of our configurations in Section 4.

The empirical performance of gradient descent on both datasets is reported in Section 4.

3.2.3 Stochastic Gradient Descent

Gradient descent (Section 3.2.2) computes the exact gradient $\nabla_{\mathcal{P}} F_\lambda(U, V)$ from every observed pair in Ω before taking a single step. As known from the Foundations of Machine Learning lectures, Stochastic gradient descent (SGD) replaces this exact gradient by a cheap estimate built from a single randomly sampled observation, and updates the factors after every sample rather than after a full pass over Ω.

Averaged objective. As already noted in the introduction to this section, SGD works with the objective scaled by $1/N_\Omega$, where $N_\Omega = |\Omega|$:

$$\bar{F}_\lambda(U, V) = \frac{1}{2N_\Omega} \sum_{(u,i)\in\Omega} \Big(U_{u,:}V_{i,:}^\top - Y_{ui}\Big)^2 + \frac{\lambda}{2N_\Omega}\left(\|U\|_F^2 + \|V\|_F^2\right).$$

This is the same objective F_λ from Equation 3.7, only divided by the number of observations, so it has the same minimizers; the scaling simply puts every step on a per-observation footing instead of a whole-dataset footing.

Per-sample gradient. For a sampled rating $(u, i) \in \Omega$, recall the local observation sets Ω_u and Ω^i from Definition 3.2, and let

$$d_u := |\Omega_u|, \qquad c_i := |\Omega^i|$$

be the number of ratings given by user u and received by movie i. Define the residual

$$e_{ui} := U_{u,:}V_{i,:}^\top - Y_{ui}.$$

Theorem 3.6 (Single-rating stochastic gradient estimator)**.** *For an observed rating $(u, i) \in \Omega$, let $g^{(\lambda)}_{(u,i)}(U, V) \in \mathcal{P}$ be the sparse vector whose only nonzero rows are*

$$g_u = e_{ui}\, V_{i,:} + \frac{\lambda}{d_u}\, U_{u,:},$$
$$g_i = e_{ui}\, U_{u,:} + \frac{\lambda}{c_i}\, V_{i,:},$$

in the U and V blocks, respectively. If (u, i) is sampled uniformly from Ω, then this estimator is unbiased:

$$\mathbb{E}\Big[g^{(\lambda)}_{(u,i)}(U, V)\Big] = \nabla_{\mathcal{P}} \bar{F}_\lambda(U, V).$$

An SGD step using this estimator updates only the corresponding user and movie rows,

$$U_{u,:} \leftarrow U_{u,:} - \eta\, g_u, \qquad V_{i,:} \leftarrow V_{i,:} - \eta\, g_i,$$

where both right-hand sides use the values of $U_{u,:}$ and $V_{i,:}$ before *either is updated.*

The factors $1/d_u$ and $1/c_i$ split the regularization gradient of each user and movie row equally among the observed ratings involving that row, which yields the correct gradient in expectation under uniform sampling.

Algorithm. Algorithm 3 states the abstract per-step update on the product space $\mathcal{P}$, mirroring how Algorithm 1 states gradient descent's update before it is expanded into row-wise form.

Algorithm 3 Stochastic gradient descent for matrix factorization

Require: Learning rate $\eta > 0$, regularization $\lambda \geq 0$, number of epochs T
Initialize $U_0 \in \mathbb{R}^{m\times r}$ and $V_0 \in \mathbb{R}^{n\times r}$ with small random entries
for $t = 0, 1, \ldots, T-1$ **do**
    $(U_{t+1}, V_{t+1}) \leftarrow (U_t, V_t)$
    for $k = 1, \ldots, N_\Omega$ **do**
        Draw $(u, i) \in \Omega$ independently and uniformly at random, with replacement
        $(U_{t+1}, V_{t+1}) \leftarrow (U_{t+1}, V_{t+1}) - \eta\, g^{(\lambda)}_{(u,i)}(U_{t+1}, V_{t+1})$
    end for
end for
return U_T, V_T

Algorithm 4 expands Algorithm 3 into the row-wise form our implementation follows.

Algorithm 4 Stochastic gradient descent (row-wise)

Require: Observed centered ratings Y_{ui} for $(u,i) \in \Omega$; local sets $(\Omega_u)_{u=1}^m$ and $(\Omega^i)_{i=1}^n$, giving counts $d_u := |\Omega_u|$ and $c_i := |\Omega^i|$; latent dimension r; regularization parameter λ; epoch count $T \in \mathbb{N}$; step size $\eta > 0$; initial factors $(U^{(0)}, V^{(0)}) \neq (0,0)$

1: **for** $t = 0, \dots, T-1$ **do**
2: $\quad (U, V) \leftarrow (U^{(t)}, V^{(t)})$
3: $\quad$ **for** $k = 1, \dots, N_\Omega$ **do**
4: $\quad\quad$ Draw $(u,i) \in \Omega$ independently and uniformly at random, with replacement
5: $\quad\quad U_{u,:}^{\text{old}} \leftarrow U_{u,:}$
6: $\quad\quad e \leftarrow U_{u,:} V_{i,:}^\top - Y_{ui}$
7: $\quad\quad U_{u,:} \leftarrow U_{u,:} - \eta \left(e\, V_{i,:} + \dfrac{\lambda}{d_u} U_{u,:} \right)$
8: $\quad\quad V_{i,:} \leftarrow V_{i,:} - \eta \left(e\, U_{u,:}^{\text{old}} + \dfrac{\lambda}{c_i} V_{i,:} \right)$
9: $\quad$ **end for**
10: $\quad (U^{(t+1)}, V^{(t+1)}) \leftarrow (U, V)$
11: **end for**
12: **return** $U^{(T)}, V^{(T)}$

Line 8 uses $U_{u,:}^{\text{old}}$, the value of $U_{u,:}$ before its update on line 7, so the two row updates within one step both use the same, pre-step pair $(U_{u,:}, V_{i,:})$, exactly as in the single-rating gradient of the previous paragraph.

As in gradient descent, the observed ratings are stored in a CSR matrix, and the SGD loops are compiled with `Numba`.

Computational complexity of our implementation. Each SGD step updates exactly one row of U and one row of V: computing the residual e is a length-r inner product, and applying both row updates requires $O(r)$ operations. One epoch performs $N_\Omega = |\Omega|$ such steps (drawing the N_Ω sample indices itself costs $O(N_\Omega)$), so one epoch costs

$$O(N_\Omega r),$$

and T epochs cost

$$O(T N_\Omega r).$$

Memory complexity of our implementation. The observed ratings and their user and movie indices require $O(N_\Omega)$ storage, the factor matrices require $O((m+n)r)$ storage, and the counts $(d_u)_{u=1}^m$ and $(c_i)_{i=1}^n$ require a further $O(m+n)$ storage. Therefore, the total memory complexity is

$$O\big(N_\Omega + (m+n)r\big).$$

Remark 3.7 (Comparison with gradient descent)**.** *Gradient descent and stochastic gradient descent minimize the same objective F_λ, but differ in what a single step sees. Gradient descent computes the exact gradient $\nabla_{\mathcal{P}} F_\lambda(U^{(t)}, V^{(t)})$ from every rating in Ω before moving, so one step corresponds to one full pass over the data. Stochastic gradient descent instead moves after every single rating, using the single-rating gradient (g_u, g_i) as an approximation of the true gradient rather than the gradient itself.*

This leads to an important practical difference. During one epoch-equivalent block, SGD performs N_Ω incremental parameter updates, whereas full-batch gradient descent performs one update after evaluating the gradient over all N_Ω observed ratings. In our experiments, SGD reached a competitive validation RMSE after substantially fewer epochs than gradient descent required iterations. Consequently, unlike gradient descent, the SGD implementation was computationally inexpensive enough to run locally and did not require the Fritz HPC cluster.

3.2.4 Alternating Least Squares

Our goal in this subsection is to derive explicit formulas for updating the factor matrices U and V in alternating least squares.

Definition 3.8 (Alternating least squares)**.** *Let $\lambda > 0$ and let $(U^{(0)}, V^{(0)})$ be an initial factor pair. ALS generates the sequence $\left(U^{(t)}, V^{(t)}\right)$ according to*

$$U^{(t+1)} \in \operatorname*{arg\,min}_{U \in \mathbb{R}^{m \times r}} F_\lambda\left(U, V^{(t)}\right), \tag{3.17}$$

$$V^{(t+1)} \in \operatorname*{arg\,min}_{V \in \mathbb{R}^{n \times r}} F_\lambda\left(U^{(t+1)}, V\right). \tag{3.18}$$

Equation 3.17 is the user half-sweep, while Equation 3.18 is the movie half-sweep.

See Hastie et al. [6, Sec. 5.4, Eqs. (56)–(57)].

Recall the local observation sets Ω_i and Ω^j from Definition 3.2. We first introduce the following notation.

Definition 3.9. *For every user i and movie j, write*

$$\boldsymbol{u}_i := U_{i,:}^\top \in \mathbb{R}^r, \qquad \boldsymbol{v}_j := V_{j,:}^\top \in \mathbb{R}^r.$$

For a fixed ordering $\Omega_i = \{j_1, \ldots, j_{d_i}\}$, where $d_i := |\Omega_i|$, define

$$V_{\Omega_i} := \begin{pmatrix} \boldsymbol{v}_{j_1}^\top \\ \vdots \\ \boldsymbol{v}_{j_{d_i}}^\top \end{pmatrix} \in \mathbb{R}^{d_i \times r}, \qquad \boldsymbol{y}_i := \begin{pmatrix} Y_{ij_1} \\ \vdots \\ Y_{ij_{d_i}} \end{pmatrix} \in \mathbb{R}^{d_i}. \tag{3.19}$$

Similarly, for a fixed ordering $\Omega^j = \{i_1, \ldots, i_{e_j}\}$, where $e_j := |\Omega^j|$, define

$$U_{\Omega^j} := \begin{pmatrix} \boldsymbol{u}_{i_1}^\top \\ \vdots \\ \boldsymbol{u}_{i_{e_j}}^\top \end{pmatrix} \in \mathbb{R}^{e_j \times r}, \qquad \boldsymbol{y}^j := \begin{pmatrix} Y_{i_1 j} \\ \vdots \\ Y_{i_{e_j} j} \end{pmatrix} \in \mathbb{R}^{e_j}. \tag{3.20}$$

For fixed one factor matrix, the corresponding ALS block problem separates into independent ridge-regression problems for the individual factor rows (see Hastie et al. [6, p. 3369]). In our notation, this gives the following decomposition.

Theorem 3.10 (Row-wise decomposition of the ALS block problems)**.** *For every fixed $V \in \mathbb{R}^{n \times r}$, the objective decomposes user-wise as*

$$F_\lambda(U, V) = \sum_{i=1}^{m} \left(\frac{1}{2} \|V_{\Omega_i} \boldsymbol{u}_i - \boldsymbol{y}_i\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{u}_i\|_2^2 \right) + \frac{\lambda}{2} \|V\|_F^2. \tag{3.21}$$

For every fixed $U \in \mathbb{R}^{m \times r}$, it decomposes movie-wise as

$$F_\lambda(U, V) = \sum_{j=1}^{n} \left(\frac{1}{2} \|U_{\Omega^j} \boldsymbol{v}_j - \boldsymbol{y}^j\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{v}_j\|_2^2 \right) + \frac{\lambda}{2} \|U\|_F^2. \tag{3.22}$$

Define $\phi_i(\boldsymbol{x}) := \frac{1}{2} \|V_{\Omega_i} \boldsymbol{x} - \boldsymbol{y}_i\|_2^2 + \frac{\lambda}{2} \|\boldsymbol{x}\|_2^2, \quad \boldsymbol{x} \in \mathbb{R}^r$. Then Equation 3.21 can be written as

$$F_\lambda(U, V) = \sum_{i=1}^{m} \phi_i(\boldsymbol{u}_i) + \frac{\lambda}{2} \|V\|_F^2. \tag{3.23}$$

Because each user factor $\boldsymbol{u}_i$ appears only in the corresponding term $\phi_i(\boldsymbol{u}_i)$, minimizing $F_\lambda(U, V)$ over U for fixed V is equivalent to minimizing the functions ϕ_i independently for every user i.

Likewise, define $\psi_j(\boldsymbol{x}) := \frac{1}{2}\|U_{\Omega^j}\boldsymbol{x} - \boldsymbol{y}^j\|_2^2 + \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2, \quad \boldsymbol{x} \in \mathbb{R}^r$. Then Equation 3.22 can be written as

$$F_\lambda(U, V) = \sum_{j=1}^{n} \psi_j(\boldsymbol{v}_j) + \frac{\lambda}{2}\|U\|_F^2. \tag{3.24}$$

Analogously, minimizing $F_\lambda(U, V)$ over V is equivalent to minimizing the functions ψ_j independently for every movie j.

To solve these independent row subproblems, we rewrite Tikhonov-regularized least-squares formula, which is given in the Foundations of Data Science lecture notes.

Theorem 3.11 (Regularized least squares)**.** *Let $A \in \mathbb{R}^{d\times r}$, $\boldsymbol{b} \in \mathbb{R}^d$, and $\lambda > 0$. Define*

$$q(\boldsymbol{x}) := \frac{1}{2}\|A\boldsymbol{x} - \boldsymbol{b}\|_2^2 + \frac{\lambda}{2}\|\boldsymbol{x}\|_2^2, \qquad \boldsymbol{x} \in \mathbb{R}^r. \tag{3.25}$$

Then q has a unique global minimizer $\boldsymbol{x}^\star$, characterized by

$$\left(A^\top A + \lambda I_r\right)\boldsymbol{x}^\star = A^\top \boldsymbol{b}. \tag{3.26}$$

The functions ϕ_i and ψ_j have exactly the form of q in Theorem 3.11. For ϕ_i, take $A = V_{\Omega_i}$ and $\boldsymbol{b} = \boldsymbol{y}_i$; for ψ_j, take $A = U_{\Omega^j}$ and $\boldsymbol{b} = \boldsymbol{y}^j$. Applying the theorem gives the following row updates.

Corollary 3.12 (Unique ALS row updates)**.** *For fixed V, the unique minimizing user factor $\boldsymbol{u}_i^+$ satisfies*

$$\left(V_{\Omega_i}^\top V_{\Omega_i} + \lambda I_r\right)\boldsymbol{u}_i^+ = V_{\Omega_i}^\top \boldsymbol{y}_i, \tag{3.27}$$

or, equivalently,

$$\left(\sum_{j\in\Omega_i} \boldsymbol{v}_j\boldsymbol{v}_j^\top + \lambda I_r\right)\boldsymbol{u}_i^+ = \sum_{j\in\Omega_i} Y_{ij}\boldsymbol{v}_j. \tag{3.28}$$

For fixed U, the unique minimizing movie factor $\boldsymbol{v}_j^+$ satisfies

$$\left(U_{\Omega^j}^\top U_{\Omega^j} + \lambda I_r\right)\boldsymbol{v}_j^+ = U_{\Omega^j}^\top \boldsymbol{y}^j, \tag{3.29}$$

or, equivalently,

$$\left(\sum_{i\in\Omega^j} \boldsymbol{u}_i\boldsymbol{u}_i^\top + \lambda I_r\right)\boldsymbol{v}_j^+ = \sum_{i\in\Omega^j} Y_{ij}\boldsymbol{u}_i. \tag{3.30}$$

The following theorem, which is an immediate consequence of Corollary 3.12 gives an argument of why we cannot initialize the ALS algorithm with both matrices U and V being zero-matrices. Note, however, that we can initialize U as a zero-matrix, since by Definition 3.8 the user half-sweep does not dependend on matrix U.

Theorem 3.13 (The zero factor pair is a fixed point of ALS)**.** *Let $\lambda > 0$. If $\left(U^{(0)}, V^{(0)}\right) = (0, 0)$, then exact ALS generates $\left(U^{(t)}, V^{(t)}\right) = (0, 0)$ for every $t \in \mathbb{N}$.*

The following Algorithm 5 is an implementation of the ALS algorithm as a consequence Theorem 3.10 using CSR and CSC matrices.and Corollary 3.12

Algorithm 5 Sparse alternating least squares

Require: Centered observed ratings Y_{ij} stored in a CSR user view and a CSC movie view; rank r; regularization parameter $\lambda > 0$; iteration count $T \in \mathbb{N}$; initialization scale $s > 0$

$U^{(0)} \leftarrow 0 \in \mathbb{R}^{m \times r}$
Initialize $V^{(0)} \in \mathbb{R}^{n \times r}$ with independent nonzero random entries of scale s
for $t = 0, \ldots, T-1$ **do**
  for all $i = 1, \ldots, m$ **do**
    $A_i \leftarrow \lambda I_r, \quad \boldsymbol{b}_i \leftarrow 0 \in \mathbb{R}^r$
    for each stored entry (j, Y_{ij}) in CSR row i **do**
      $A_i \leftarrow A_i + \boldsymbol{v}_j^{(t)} \left(\boldsymbol{v}_j^{(t)}\right)^\top$
      $\boldsymbol{b}_i \leftarrow \boldsymbol{b}_i + Y_{ij} \boldsymbol{v}_j^{(t)}$
    end for
    $\boldsymbol{u}_i^{(t+1)} \leftarrow \mathrm{solve}(A_i, \boldsymbol{b}_i)$
  end for
  for all $j = 1, \ldots, n$ **do**
    $C_j \leftarrow \lambda I_r, \quad \boldsymbol{d}_j \leftarrow 0 \in \mathbb{R}^r$
    for each stored entry (i, Y_{ij}) in CSC column j **do**
      $C_j \leftarrow C_j + \boldsymbol{u}_i^{(t+1)} \left(\boldsymbol{u}_i^{(t+1)}\right)^\top$
      $\boldsymbol{d}_j \leftarrow \boldsymbol{d}_j + Y_{ij} \boldsymbol{u}_i^{(t+1)}$
    end for
    $\boldsymbol{v}_j^{(t+1)} \leftarrow \mathrm{solve}(C_j, \boldsymbol{d}_j)$
  end for
end for
return $U^{(T)}, V^{(T)}$

CSR and CSC are particularly suitable for ALS. In CSR format, the sparse matrix is represented by three arrays: `data`, `indices`, and `indptr`. For user row i, the entries with positions $p \in [\texttt{indptr}[i], \texttt{indptr}[i+1])$ are exactly its observed ratings. At such a position p, `data`$[p]$ stores the centered rating Y_{ij} and `indices`$[p]$ stores its movie index j. That is, the CSR provides direct acess to all observed movies $\Omega_i = \{j : (i,j) \in \Omega\}$ of user i.

The CSC representation is analogous, but organized by movie columns. For movie j, the positions $p \in [\texttt{indptr}[j], \texttt{indptr}[j+1])$ contain exactly the observed ratings of this movie. At such a position p, `csc_data`$[p]$ stores the centered rating Y_{ij} and `csc_indices`$[p]$ stores the corresponding user index i. Similarly, CSC provides direct access to all observed users $\Omega^j = \{i : (i,j) \in \Omega\}$ who rated a fixed movie j.

Numba implementation. The user and movie sweeps are compiled with `Numba`. The compiled kernels operate directly on the CSR and CSC arrays discussed above and perform the row updates from Algorithm 5.

The outer user and movie loops, whose row updates are independent within each half-sweep, are parallelized with Numba's `prange`.

Computational complexity. Let $N_\Omega := |\Omega|$ be the number of observed ratings and let $d_i := |\Omega_i|$ be the number of ratings of user i. For every observed pair (i,j), the user sweep adds the rank-one matrix $\boldsymbol{v}_j \boldsymbol{v}_j^\top$ to A_i. This outer product has r^2 entries and therefore costs $\mathcal{O}(r^2)$ operations. The corresponding update $\boldsymbol{b}_i \leftarrow \boldsymbol{b}_i + Y_{ij}\boldsymbol{v}_j$ costs $\mathcal{O}(r)$. Hence the contribution of one observed rating is $\mathcal{O}(r^2 + r) = \mathcal{O}(r^2)$, and accumulating A_i and $\boldsymbol{b}_i$ for user i costs $\mathcal{O}(d_i r^2)$.

Then, solving dense $r \times r$ system $A_i \boldsymbol{u}_i = \boldsymbol{b}_i$, which costs $\mathcal{O}(r^3)$. Thus, one user update costs

$$\mathcal{O}\left(d_i r^2 + r^3\right).$$

The initialization of A_i and $\boldsymbol{b}_i$ costs at most $\mathcal{O}(r^2)$ and is dominated by this bound. Summing the update cost over all m users gives because $\sum_{i=1}^{m} d_i = N_\Omega$:

$$\mathcal{O}\left(\sum_{i=1}^{m}\left(d_i r^2 + r^3\right)\right) = \mathcal{O}\left(r^2 \sum_{i=1}^{m} d_i + m r^3\right) = \mathcal{O}\left(N_\Omega r^2 + m r^3\right).$$

For the movie half-sweep, let $e_j := |\Omega^j|$. The same argument gives a cost of $\mathcal{O}(e_j r^2 + r^3)$ for movie j. Since $\sum_{j=1}^{n} e_j = N_\Omega$, the complete movie half-sweep costs

$$\mathcal{O}\left(N_\Omega r^2 + n r^3\right).$$

Consequently, one full ALS iteration, consisting of both half-sweeps, costs

$$\mathcal{O}\left(N_\Omega r^2 + (m+n) r^3\right), \tag{3.31}$$

and T iterations require

$$\mathcal{O}\left(T\left(N_\Omega r^2 + (m+n) r^3\right)\right). \tag{3.32}$$

Memory complexity. The CSR and CSC representations each store the N_Ω centered ratings and their indices, together with pointer arrays of lengths $m+1$ and $n+1$. All sparse rating data therefore require $\mathcal{O}(N_\Omega + m + n)$ memory. The factor matrices require $\mathcal{O}((m+n)r)$ memory. During one row update, the algorithm additionally stores one $r \times r$ normal matrix and one length-r right-hand side, which require $\mathcal{O}(r^2)$ temporary memory. Therefore, the total memory complexity is

$$\mathcal{O}\left(N_\Omega + (m+n) r + r^2\right).$$

4 Results

The data was first split into training and test sets in an 80/20 ratio. For hyperparameter selection, the training set was further split into a training-validation set and a validation set, again in an 80/20 ratio. After selecting the hyperparameters, each model was retrained on the complete original training set and evaluated once on the held-out test set.

4.1 `ml-latest-small`

4.1.1 Hard Thresholding

A coarse search over $r \in \{2, 3, 4\}$ for 40 iterations selected $r = 2$ based on validation RMSE. Since the validation error was still decreasing, the run with $r = 2$ was extended to 60 iterations. Further improvement was small relative to the cost of additional dense SVD computations; therefore, the final configuration was $r = 2$ and $T = 60$.

After retraining this configuration on all 80,668 training ratings, hard thresholding obtained a test RMSE of **0.887721** and a test MAE of **0.682578**.

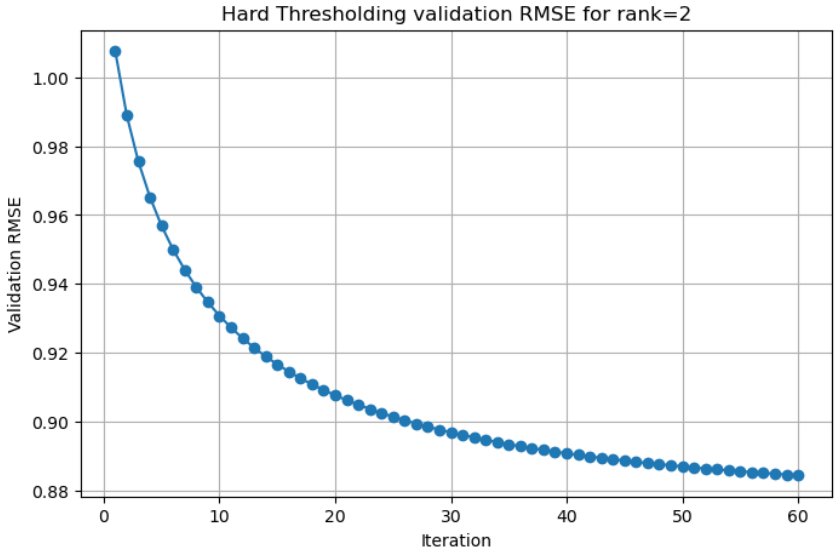


Figure 3: Validation RMSE of hard thresholding with $r = 2$. The error continues to decrease over 60 iterations, although the improvement becomes progressively smaller.

4.1.2 Soft Thresholding

A coarse, and then fine, search over τ values selected $\tau = 11$ based on validation RMSE. A 100-iteration validation run with $\tau = 11$ then selected iteration 86 as the stopping point. The final configuration was therefore $\tau = 11$ and $T = 86$.

After retraining this configuration on the complete training set, soft thresholding produced a test RMSE of **0.870291** and a test MAE of **0.677180**.

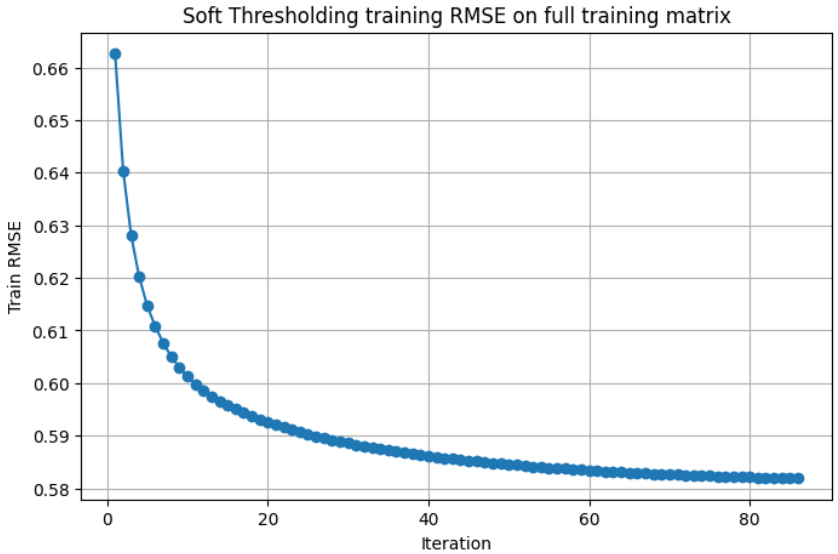


Figure 4: Training RMSE of the final soft-thresholding model on the complete `ml-latest-small` training set with $\tau = 11$. The error decreases smoothly over 86 iterations, while the improvement becomes gradually smaller.

4.1.3 Hard versus Soft Thresholding

Hard thresholding fixes how many singular components may remain and leaves their values unchanged. Soft thresholding instead fixes the amount of shrinkage: weak components disappear,

while all surviving components are reduced. Soft thresholding produced the lower prediction error in this experiment.

Property	Global mean	Hard	Soft
Low-rank control	none	$\mathrm{rank}(X) \le r$	$\lambda\|X\|_*$
Selected setting	–	$r = 2$, $T = 60$	$\tau = 11$, $T = 86$
Test RMSE	1.048841	0.887721	**0.870291**
Test MAE	0.831591	0.682578	**0.677180**
RMSE reduction	–	15.4 %	**17.0 %**

Table 3: Final comparison on `ml-latest-small`.

The numerical advantage of soft thresholding is modest but consistent. Its test RMSE is 0.017430 lower than that of hard thresholding, and its test MAE is lower by 0.005398. Both models also improve clearly over the global-mean baseline, showing that the low-rank structure captures information that a single constant prediction cannot represent.

Hard thresholding stops after 60 iterations, whereas soft thresholding uses 86 iterations. Because both small-data implementations perform one dense SVD per iteration on matrices of the same size, soft thresholding performs approximately

$$\frac{86}{60} \approx 1.43$$

times as many SVD computations.

Both implementations require $O(mn)$ memory. Their dominant computational costs under the selected settings are approximately $O(60mnq)$ for hard thresholding and $O(86mnq)$ for soft thresholding. Hard thresholding is therefore computationally cheaper under the tested settings, while soft thresholding provides the better test RMSE and MAE.

4.1.4 Gradient Descent

A coarse grid search followed by a refined search selected rank $r = 50$, learning rate $\eta = 0.003$, regularization $\lambda = 8$, and 140 iterations; initialization scale 0.05 was fixed. This configuration achieved a validation RMSE of 0.9040. After retraining on the full training set, the model obtained a test RMSE of **0.9044** and a test MAE of **0.6952**.

As shown inFigure 5, both the training objective and the training RMSE decreased smoothly throughout the optimization.

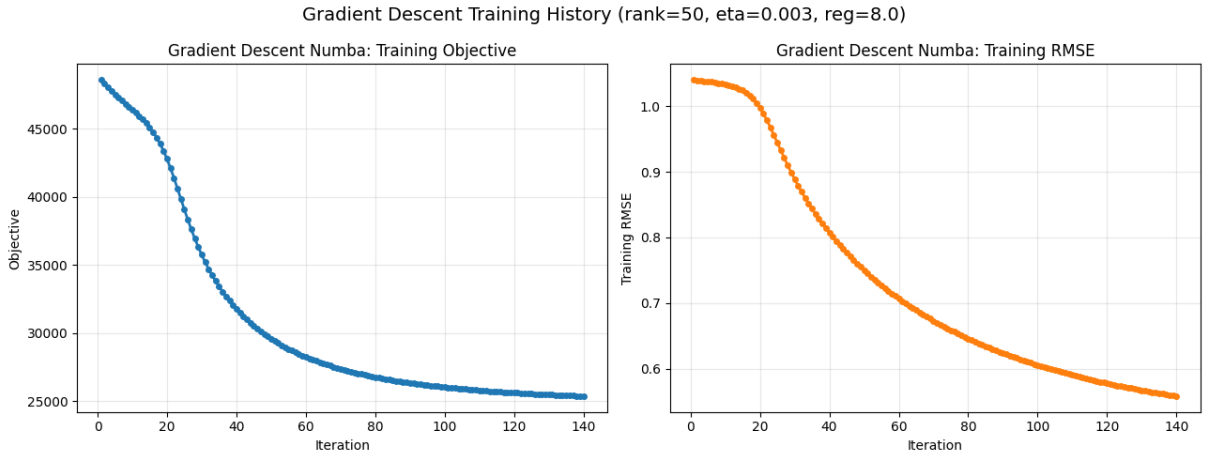


Figure 5: Training objective and training RMSE of the final gradient descent model on `ml-latest-small`. Both quantities decrease smoothly, indicating stable optimization.

4.1.5 Stochastic Gradient Descent

A coarse grid search followed by a refined search selected rank $r = 80$, learning rate $\eta = 0.0075$, regularization $\lambda = 4.0$, 49 iterations, and initialization scale 0.003. This configuration achieved a validation RMSE of 0.8898. After retraining on the full training set, the model obtained a test RMSE of **0.8925** and a test MAE of **0.6818**.

As shown in Figure 6, the training objective and training RMSE remain almost constant for the first ten epochs before decreasing steadily for the remainder of training. This behavior is consistent with the small initialization scale.

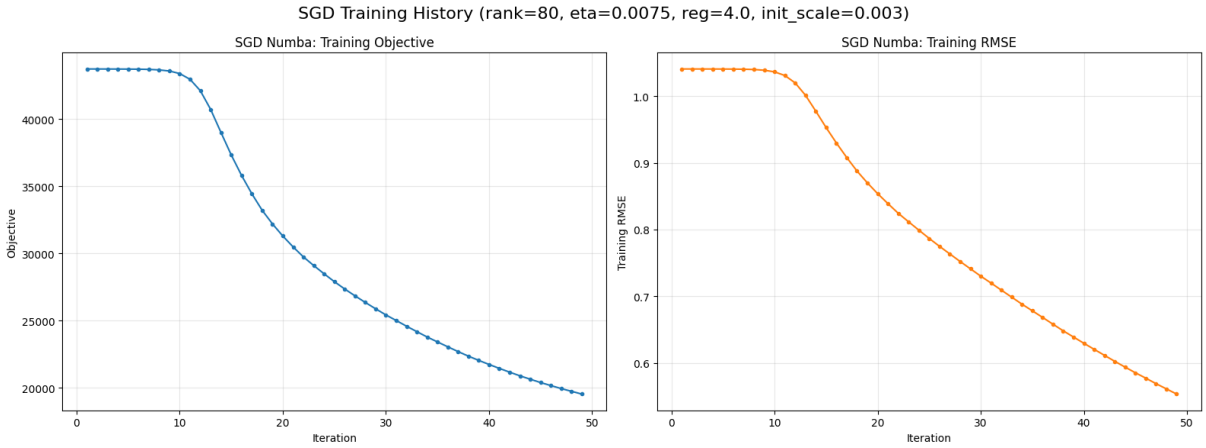


Figure 6: Training objective and training RMSE of the final stochastic gradient descent model on `ml-latest-small` ($r = 80$, $\eta = 0.0075$, $\lambda = 4.0$). Both quantities stay almost flat for about ten epochs before decreasing steadily for the remaining epochs.

4.1.6 Alternating Least Squares

A coarse and then refined grid search selected latent rank $r = 2$ and regularization $\lambda = 4$. The initialization scale was fixed at 0.01 in all runs. The lowest validation RMSE, 0.8952, was reached after 23 full ALS iterations. Retraining this configuration on the full training set gave a test RMSE of **0.8966** and a test MAE of **0.6868**.

Figure 7 shows a sharp decrease in both the training objective and training RMSE during the first few iterations. Both quantities then level off, with no visible instability.

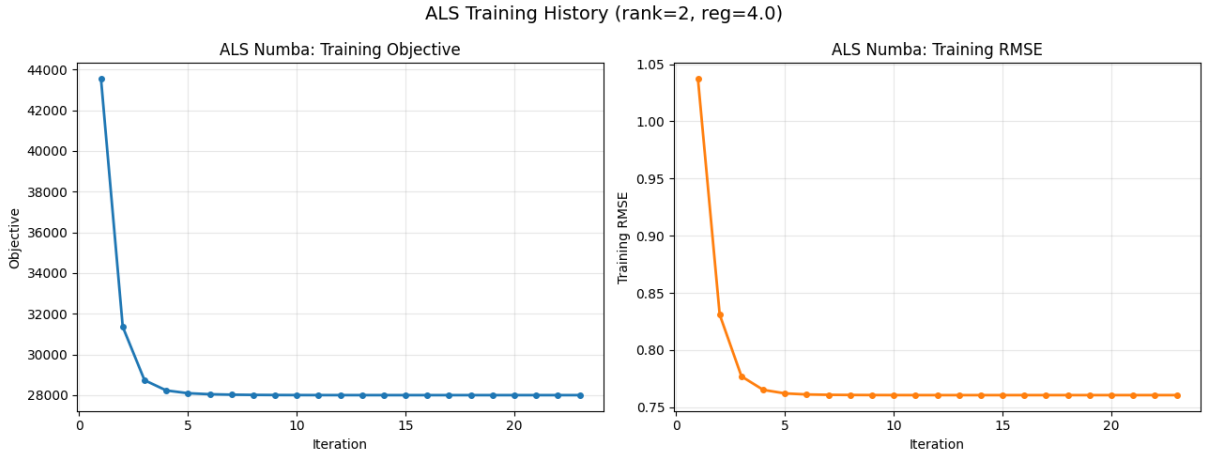


Figure 7: Training objective and training RMSE of ALS on `ml-latest-small` with $r = 2$ and $\lambda = 4$. Both quantities decrease rapidly and then stabilize.

4.1.7 Model Comparison

Figure 8 compare all models on the held-out `ml-latest-small` test set. Soft thresholding achieved the lowest error on both metrics, with an RMSE of 0.870 and an MAE of 0.677. The results of SGD and ALS were comparable. However, SGD achieved lower test errors than GD. Our experiment does not identify a cause for this difference. Every matrix-completion model improved substantially over the global-mean baseline.

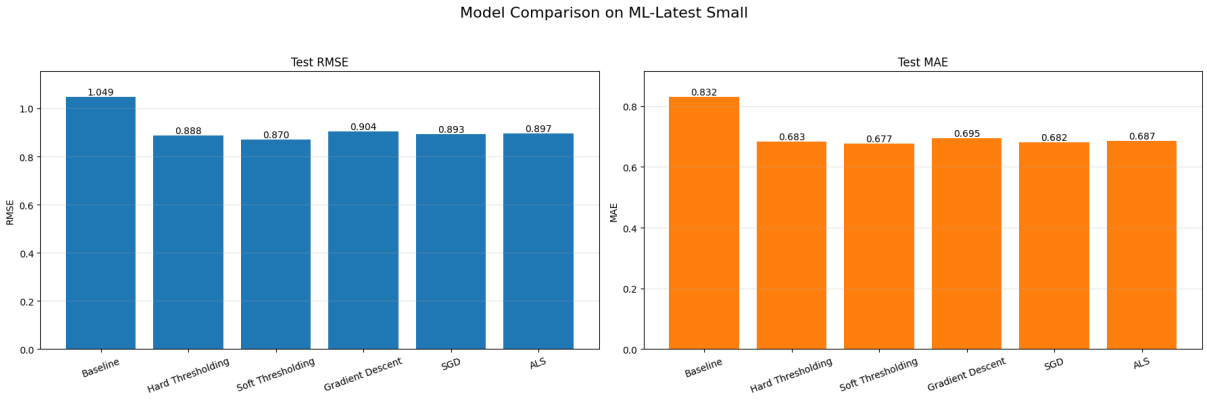


Figure 8: Test RMSE and MAE comparison of all models on `ml-latest-small`.

Parameters on `ml-latest-small`.

Model	Rank	τ	η	λ	Init. scale	Iterations
Hard Thresholding	2	–	–	–	–	60
Soft Thresholding	–	11	–	–	–	86
Gradient Descent	50	–	0.003	8.0	0.05	140
SGD	80	–	0.0075	4.0	0.003	49
ALS	2	–	–	4.0	–	23

4.2 `ml-latest`

4.2.1 Hard Thresholding

Hyperparameter tuning was performed on the Fritz HPC cluster. We performed a coarse search for the rank parameter and multiple fine-graned searches, producing the results of $r = 12$ and 450 number of iteration. The number of iterations was selected differently to other models. Normally, we run hyperparameter searches with a sufficient margin of iterations, and then select the iteration with the best validation RMSE. In this case, the validation RMSE continued to decrease throughout the whole run and reached its lowest value at iteration 450. But this improvement was only miniscule, as can be seen in Table 4. So we decided not to extend the number of iteratins further, and chose 450 as our final parameter. This choice yielded a validation RMSE 0.809475.

Rank	Selected iterations	Validation RMSE	Last-50 decrease
12	450	0.809475	0.000151

Iteration	Training RMSE	Validation RMSE
350	0.716026	0.809900
400	0.713416	0.809626
420	0.712495	0.809554
430	0.712056	0.809524
440	0.711632	0.809498
450	**0.711220**	**0.809475**

Table 4: Selected hard-thresholding configuration and late-stage convergence. The last-50 decrease is the reduction in validation RMSE between iterations 400 and 450. The lower table shows selected checkpoints from the final 100 iterations.

After retraining on the complete training set for 450 iterations, the final model achieved a test RMSE of **0.7996** and a test MAE of **0.5986**.

As shown in Figure 9, the training RMSE decreases smoothly throughout the final run, with progressively smaller improvements toward the end of training.

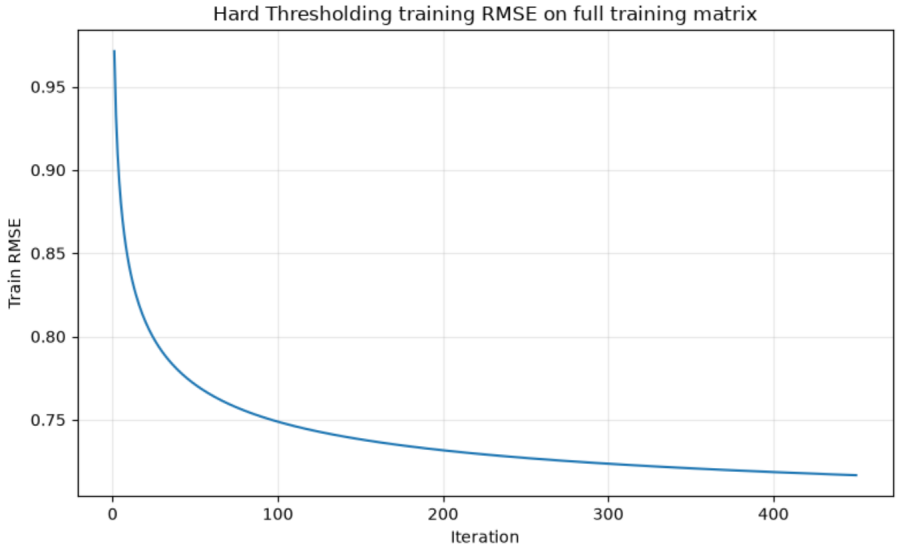


Figure 9: Training RMSE of the final hard-thresholding model on the complete `ml-latest` training set with $r = 12$. The error decreases smoothly over 450 iterations, while the improvement becomes gradually smaller.

4.2.2 Soft Thresholding

Soft thresholding was evaluated only on `ml-latest-small`. Although a scalable implementation was completed for hard thresholding, applying soft thresholding to `ml-latest` would have required a separate adaptation of the existing code to the Fritz HPC environment, followed by a new search for τ and the stopping iteration. Due to the technical complexity of this work and the available project time, this extension was not implemented. Therefore, no full-dataset results are reported for soft thresholding.

4.2.3 Gradient Descent

On our local machine, the resulting runtime and memory pressure made the complete search impractical, so parameter tuning was performed on the Fritz HPC cluster. Three-stage grid search selected latent rank $r = 60$, regularization $\lambda = 1$, and 177 retained iterations; initialization scale 0.01 was fixed. On the 80% training-fit split, the best learning rate was $\eta = 2.5 \times 10^{-4}$, yielding a validation RMSE of 0.8107.

After retraining on the full training set with the same learning rate, the training objective and training RMSE showed pronounced spikes after approximately 160 iterations. This indicated that the selected step size was too large.

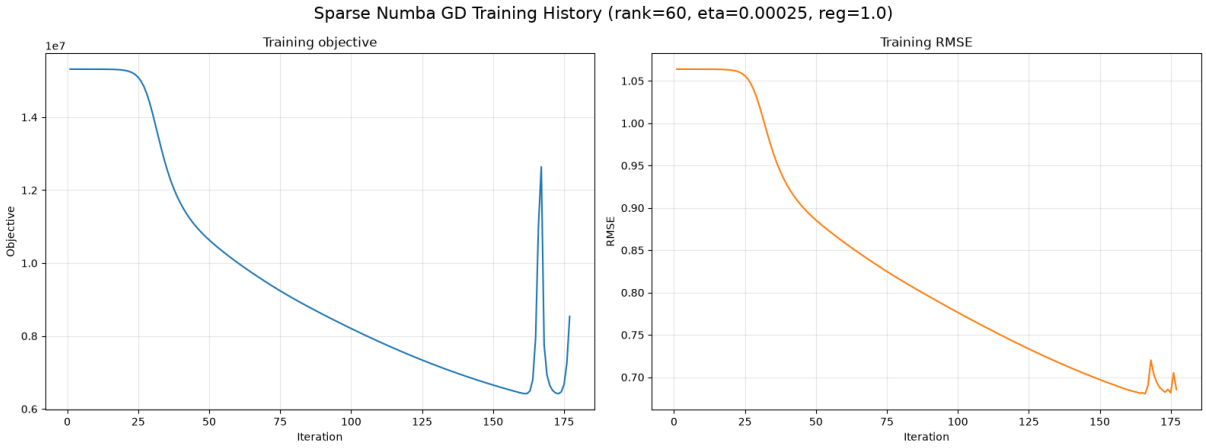


Figure 10: Gradient descent training on the full `ml-latest` training set using the learning rate selected on the training-fit split, $\eta = 2.5 \times 10^{-4}$. The objective and training RMSE become unstable after approximately 160 iterations.

In our implementation, the gradient contributions of individual ratings are accumulated by summation rather than averaged. Consequently, increasing the number of training observations changes the scale of the gradient. Since the full-data run became unstable, we applied the following heuristic: we reduced the learning rate by $N_{\text{fit}}/N_{\text{train}} = 0.8$, resulting in $\eta = 2.0 \times 10^{-4}$. It is important to note that this adjustment was based only on the training set, so there was no data leakage.

With the rescaled learning rate, both the training objective and the training RMSE decreased smoothly.

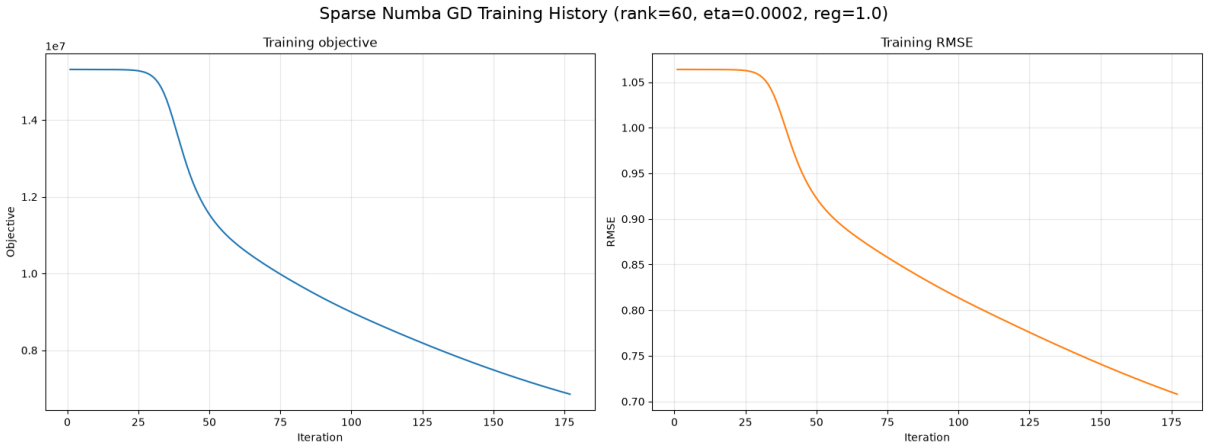


Figure 11: Final gradient descent training on the full `ml-latest` training set with the rescaled learning rate $\eta = 2.0 \times 10^{-4}$. Both the objective and the training RMSE decrease smoothly.

The final model achieved a test RMSE of **0.8007** and a test MAE of **0.6000**.

4.2.4 Stochastic Gradient Descent

A coarse grid search followed by a refined search on the training-fit split selected rank $r = 15$, learning rate $\eta = 0.012$, and regularization $\lambda = 2.5$; initialization scale 0.003 was fixed. Training this configuration for 30 epochs and monitoring the validation RMSE after every epoch identified epoch 14 as the best point, with a validation RMSE of 0.8104. After retraining on the full training set for 14 epochs, the model obtained a test RMSE of **0.7991** and a test MAE of **0.5990**.

During the final 14-epoch training run, both the training objective and the training RMSE decreased smoothly, as shown in Figure **??**.

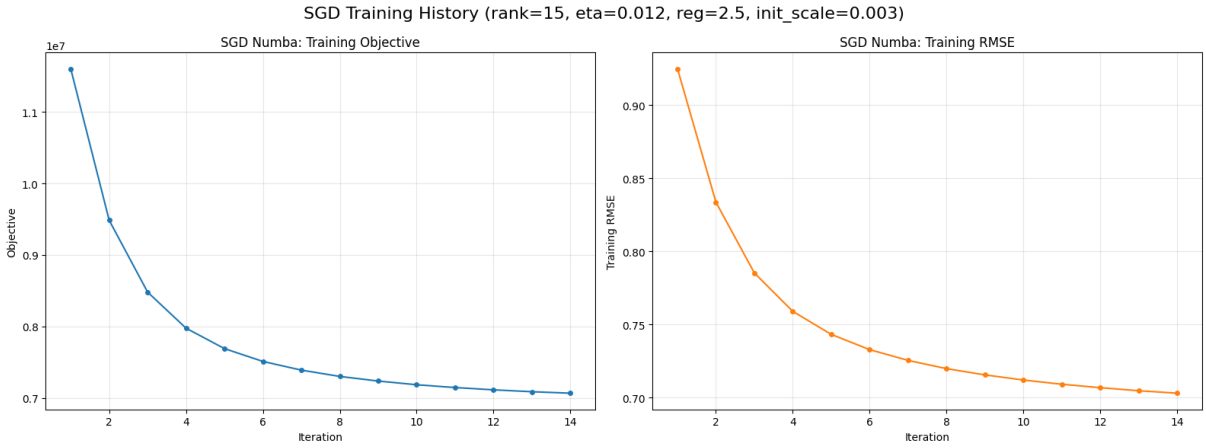


Figure 12: Training objective and training RMSE of the final stochastic gradient descent model on `ml-latest` ($r = 15$, $\eta = 0.012$, $\lambda = 2.5$). Both quantities decrease sharply over the first few epochs and then level off.

4.2.5 Alternating Least Squares

For `ml-latest`, hyperparameter selected $r = 15$ and $\lambda = 10$. The initialization scale was fixed at 0.01 in all runs. The best validation RMSE was 0.8036, obtained after 57 full ALS iterations. After retraining on the complete training set for 57 iterations, the model achieved a test RMSE of **0.7931** and a test MAE of **0.5936**.

As shown in Figure 13, the objective and training RMSE decrease rapidly at the start of training and then improve more gradually.

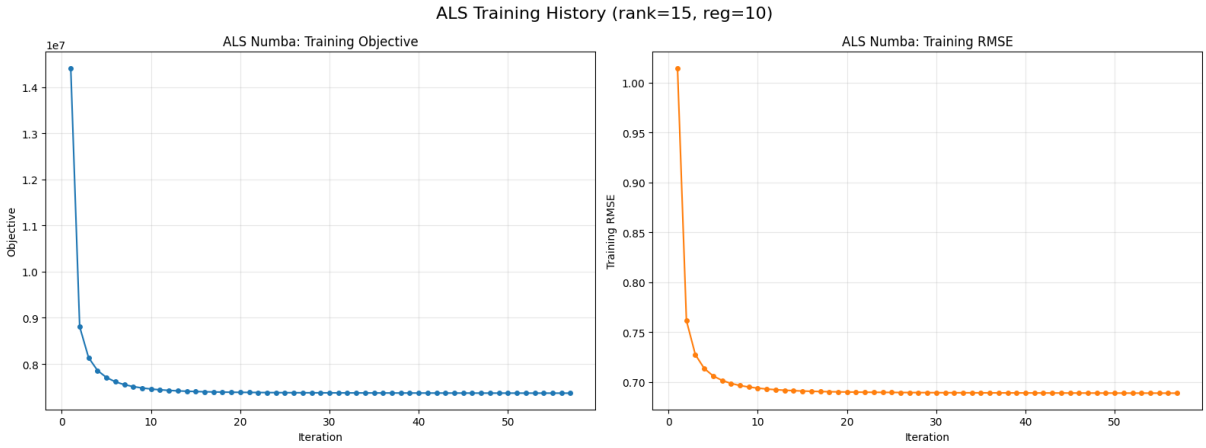


Figure 13: Training objective and training RMSE of ALS on `ml-latest` with $r = 15$ and $\lambda = 10$. Both quantities decrease smoothly over the 57 iterations.

4.2.6 Model Comparison

Figure 14 summarize the final results on `ml-latest`. ALS obtained the lowest test error. SGD, GD and hard thresholding were very similar. As on `ml-latest-small` data set, SGD achieved better results than GD, although the difference was smaller on the large data set. All matrix-completion methods clearly outperformed the baseline.

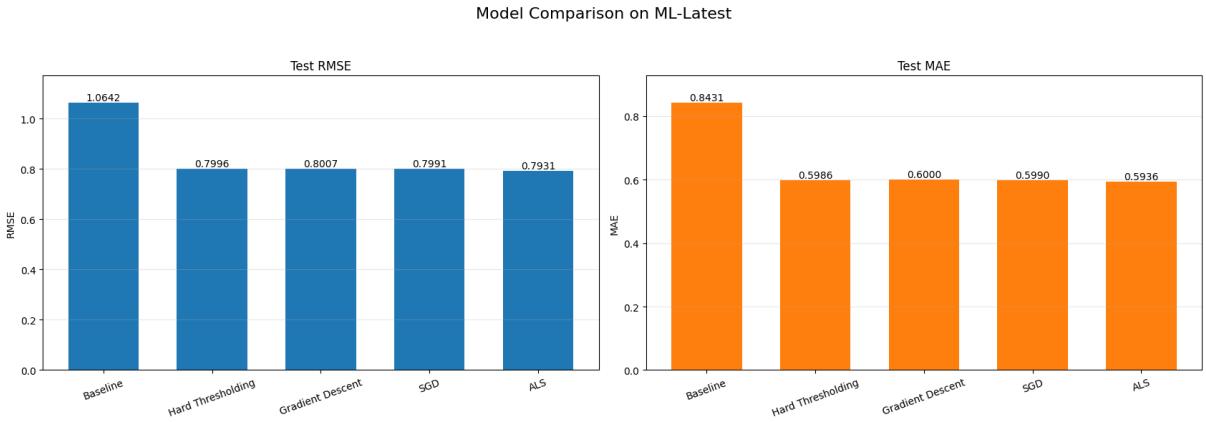


Figure 14: Test RMSE and MAE comparison of the models on `ml-latest`.

Parameters on `ml-latest`.

Model	Rank	τ	η	λ	Init. scale	Iterations
Hard Thresholding	12	–	–	–	–	450
Gradient Descent	60	–	0.0002	1.0	0.01	177
SGD	15	–	0.012	2.5	0.003	14
ALS	15	–	–	10.0	–	57

5 Difficulties

Our biggest recurring problem was runtime. Several of our models, in particular the sparse SGD and ALS implementations on `ml-latest`, were initially far too slow to tune with a proper hyperparameter search. We were able to fix this only partially: compiling the inner training loops with `numba` and, for ALS, parallelizing the per-row solves brought runtimes down substantially, but even after these optimizations some configurations still took hours to train, which limited how exhaustively we could search the hyperparameter space at full scale. Hard and soft thresholding

were affected even more: as discussed previously, due to Their use of dense matrices, we could not run them on our personal computers on `ml-latest`.

We additionally ran into difficulties with the HPC cluster which we used to train GD and Hard Thresholding. Simply getting the cluster set up and usable for our workflow was a significant hurdle on its own, and adjusting our implementation to run correctly in that environment took additional effort. Even after this setup work, some training runs on the cluster still took a long time to complete.

Finally, we had originally planned to accompany our point predictions with confidence intervals, to give a sense of how uncertain each predicted rating is. We ultimately had to drop this, as understanding the required theory and implementing the method correctly would have required more time than we had remaining.

6 Contributions

This project was a collaborative effort, and all team members contributed to the data exploration, the modeling work, and the writing of this report. Within this shared effort, each team member took the lead on specific technical areas:

- **Labeat Dobraj** focused primarily on data acquisition and preprocessing, including downloading the MovieLens datasets and preparing the data for the subsequent modeling steps, as well as the Stochastic Gradient Descent model.
- **Oleksandr Bobryshev** focused primarily on the matrix-factorization models based on Gradient Descent and Alternating Least Squares.
- **Nikita Mykytenko** focused primarily on the spectral matrix-completion models based on soft and hard thresholding.

Despite these individual focus areas, all major decisions – including the design of the experimental methodology and the interpretation of results – were discussed and made jointly by the entire team.

7 Use of Artificial Intelligence

Generative AI was used as a supporting tool during the project. It assisted with visualization code, repetitive programming tasks, and troubleshooting. All suggested code was reviewed and tested before use.

For this report, AI was used to correct English grammar and improve the clarity and logical flow of the text. It was not used as a scientific source. All methods, experiments, results, and conclusions were reviewed and verified by the authors.

References

[1] M. A. Davenport and J. Romberg. "An Overview of Low-Rank Matrix Recovery from Incomplete Observations". In: *arXiv preprint arXiv:1601.06422* (2016). URL: `https://arxiv.org/abs/1601.06422`.

[2] D. Stöger. "Foundations of Data Science". Lecture notes, Catholic University of Eichstätt–Ingolstadt. 2026.

[3] Y. Chi, Y. M. Lu, and Y. Chen. "Nonconvex Optimization Meets Low-Rank Matrix Factorization: An Overview". In: *IEEE Transactions on Signal Processing* 67.20 (2019), pp. 5239–5269. DOI: `10.1109/TSP.2019.2937282`. URL: `https://doi.org/10.1109/TSP.2019.2937282`.

[4] N. Boumal. *An Introduction to Optimization on Smooth Manifolds*. Cambridge University Press, 2023. DOI: 10.1017/9781009166164.

[5] A. Beck. *First-Order Methods in Optimization*. MOS-SIAM Series on Optimization 25. Society for Industrial and Applied Mathematics, 2017. DOI: 10.1137/1.9781611974997.

[6] T. Hastie, R. Mazumder, J. D. Lee, and R. Zadeh. "Matrix Completion and Low-Rank SVD via Fast Alternating Least Squares". In: *Journal of Machine Learning Research* 16.104 (2015), pp. 3367–3402. URL: https://www.jmlr.org/papers/v16/hastie15a.html.